

TOP 100 MOST VVI LLM & GENERATIVE AI INTERVIEW QUESTIONS (LLM Engineering)

SECTION 1 — LLM FUNDAMENTALS (Most Repeated)

1. What is Generative AI?

Generative AI refers to AI systems capable of creating new content such as text, images, code, audio, or video by learning patterns from large datasets. Unlike traditional ML models that classify or predict, generative models produce novel outputs. Models like GPT generate text token by token using probability distributions learned during training. It is widely used in chatbots, coding assistants, and content generation systems.

2. What is an LLM (Large Language Model)?

A Large Language Model is a deep learning model trained on massive text datasets using transformer architecture to understand and generate human-like language. It learns linguistic patterns, reasoning structures, and contextual relationships between words. LLMs operate using tokens instead of raw words. Examples include GPT, LLaMA, Claude, and Gemini.

3. How does an LLM generate text?

LLMs generate text autoregressively, meaning they predict the next token based on previous tokens. The model calculates probability distributions over vocabulary using attention mechanisms. The highest-probability token (or sampled token) is selected repeatedly until completion. This process allows coherent sentence formation.

☒ 4. What is a Token in LLMs?

A token is the smallest unit processed by an LLM, which can be a word, subword, or character fragment. Tokenization converts text into numerical representations understandable by neural networks. Different models use different tokenizers like BPE or SentencePiece. Token count directly impacts cost and context length.

☒ 5. What is Transformer Architecture?

Transformer is a neural network architecture introduced in *Attention Is All You Need*. It replaces recurrence with self-attention mechanisms. Transformers process sequences in parallel, making training faster and scalable. Core components include attention layers, feed-forward networks, and positional encoding.

☒ 6. What is Self-Attention?

Self-attention allows a model to weigh the importance of different words in a sentence relative to each other. Each token attends to all other tokens to understand context. This enables capturing long-range dependencies better than RNNs. It is the backbone of modern LLMs.

☒ 7. Why are Transformers better than RNN/LSTM?

Transformers allow parallel computation, reducing training time significantly. They handle long dependencies without vanishing gradients. Attention mechanisms capture global context efficiently. This makes them scalable to billions of parameters.

☒ 8. What are Parameters in LLM?

Parameters are learnable weights inside the neural network adjusted during training. They store linguistic knowledge learned from data. Larger parameter counts generally

improve reasoning ability but increase compute cost. GPT-4-scale models contain billions or trillions of parameters.

9. What is Pretraining?

Pretraining is the initial training phase where an LLM learns general language patterns using massive unlabeled text via self-supervised learning. The model predicts missing or next tokens. This phase builds foundational knowledge before specialization.

10. What is Fine-Tuning?

Fine-tuning adapts a pretrained model to specific tasks or domains using smaller labeled datasets. It improves task accuracy without training from scratch. Examples include instruction tuning and domain adaptation.

SECTION 2 — TRAINING & LEARNING (Highly Repeated)

11. What is Self-Supervised Learning?

Self-supervised learning generates labels from data itself. In LLMs, the model predicts next tokens as supervision. This removes the need for manual labeling. It enables training on internet-scale datasets.

12. What is Instruction Tuning?

Instruction tuning trains models on instruction-response pairs so they follow human commands effectively. It improves usability and conversational alignment. Models learn to interpret prompts as tasks rather than plain text.

☒ 13. What is RLHF (Reinforcement Learning from Human Feedback)?

RLHF improves model alignment using human preference rankings. Humans rate outputs, and a reward model learns preferred responses. The LLM is optimized using reinforcement learning. This reduces harmful or irrelevant responses.

☒ 14. What is Alignment in LLMs?

Alignment ensures model outputs match human values, safety expectations, and user intent. Techniques include RLHF, safety filtering, and policy constraints. Without alignment, models may produce biased or unsafe responses.

☒ 15. What is Overfitting in LLM Fine-Tuning?

Overfitting occurs when the model memorizes training examples instead of learning general patterns. It reduces performance on unseen data. Regularization and diverse datasets help prevent it.

☒ 16. What is Temperature in Text Generation?

Temperature controls randomness in token selection. Low temperature produces deterministic outputs, while high temperature increases creativity. It adjusts probability distribution sharpness during sampling.

☒ 17. What is Top-K Sampling?

Top-K sampling restricts token selection to the K most probable tokens. This prevents unlikely outputs while maintaining diversity. It balances randomness and coherence.

☒ 18. What is Top-P (Nucleus Sampling)?

Top-P sampling selects tokens whose cumulative probability exceeds threshold P. It dynamically adjusts candidate tokens. This often produces more natural text than Top-K.

☒ 19. What is Beam Search?

Beam search explores multiple candidate sequences simultaneously and selects the best scoring output. It improves accuracy but reduces diversity compared to sampling.

☒ 20. What is Context Window?

Context window is the maximum number of tokens an LLM can process at once. Larger context enables better long-document reasoning. It is limited by memory and computation constraints.

★ SECTION 3 — EMBEDDINGS & VECTOR DATABASE (VERY FREQUENT)

☒ 21. What are Embeddings?

Embeddings are dense numerical vector representations capturing semantic meaning of text. Similar texts have similar vectors. They enable semantic search and retrieval.

☒ 22. Why are Embeddings Important?

They allow similarity comparison using vector distance metrics. Embeddings power search engines, recommendation systems, and RAG pipelines.

☒ 23. What is Cosine Similarity?

Cosine similarity measures angle similarity between two vectors. Values closer to 1 indicate higher semantic similarity. It is commonly used in embedding retrieval.

☒ 24. What is a Vector Database?

A vector database stores embeddings and enables fast similarity search. Examples include Pinecone, FAISS, Weaviate, and ChromaDB.

☒ 25. Why not use SQL database for embeddings?

Traditional databases are inefficient for high-dimensional similarity search. Vector DBs use ANN indexing optimized for nearest-neighbor queries.

★ SECTION 4 — RAG (MOST ASKED IN 2024–2026)

☒ 26. What is RAG (Retrieval Augmented Generation)?

RAG combines retrieval systems with LLMs. Relevant documents are retrieved from a knowledge base and injected into prompts. This improves factual accuracy and reduces hallucinations.

☒ 27. Why is RAG preferred over Fine-Tuning?

RAG updates knowledge without retraining models. It is cheaper, scalable, and keeps data private. Fine-tuning modifies weights, while RAG modifies context.

☒ 28. RAG Pipeline Steps?

Document ingestion → chunking → embedding → vector storage → query embedding → retrieval → prompt augmentation → generation.

☒ 29. What is Chunking?

Chunking splits large documents into smaller pieces before embedding. Proper chunk size improves retrieval quality and context relevance.

☒ 30. What is Hybrid Search?

Hybrid search combines keyword search and vector similarity search. It improves accuracy by mixing semantic and lexical matching.

★ SECTION 5 — PROMPT ENGINEERING (VERY VERY COMMON)

☒ 31. What is Prompt Engineering?

Prompt engineering is designing structured inputs to guide LLM behavior effectively. It includes instructions, examples, constraints, and formatting.

☒ 32. Zero-shot vs Few-shot Prompting?

Zero-shot provides no examples. Few-shot includes sample demonstrations to guide output style and reasoning.

☒ 33. What is Chain-of-Thought Prompting?

It encourages the model to reason step-by-step before answering. This improves reasoning tasks like math and logic.

☒ 34. What is System Prompt?

System prompt defines model behavior and personality globally in chat systems.

☒ 35. What is Prompt Injection?

Prompt injection is a security attack where malicious input manipulates model behavior. Mitigation includes validation and context isolation.

★ SECTION 6 — HALLUCINATION & SAFETY

☒ 36. What is Hallucination in LLM?

Hallucination occurs when models generate incorrect but confident information. It happens due to probabilistic generation rather than factual verification.

☒ 37. How to Reduce Hallucination?

Use RAG, grounding, structured prompts, verification pipelines, and lower temperature settings.

☒ 38. What is Guardrails?

Guardrails are constraints or filters ensuring safe and controlled outputs.

☒ 39. What is Model Evaluation in LLM?

Evaluation measures quality using metrics like BLEU, ROUGE, human evaluation, and task-specific benchmarks.

☒ 40. Offline vs Online Evaluation?

Offline uses datasets; online evaluates real user interactions.

★ SECTION 7 — AGENTIC AI (HOT INTERVIEW TOPIC)

☒ 41. What is an AI Agent?

An AI agent uses LLM reasoning plus tools, memory, and planning to perform tasks autonomously.

☒ 42. Components of AI Agent?

LLM brain, tools, memory, planner, execution loop.

☒ 43. What is Tool Calling?

LLM decides when to call external APIs or functions to complete tasks.

☒ 44. What is ReAct Framework?

Reason + Act loop where model alternates reasoning and actions.

☒ **45. What is Multi-Agent System?**

Multiple agents collaborate, specialize, and communicate to solve complex tasks.

★ SECTION 8 — LLM DEPLOYMENT (INTERVIEW FAVORITE)

☒ **46. How to deploy an LLM app?**

Backend API + model inference + vector DB + frontend + monitoring pipeline.

☒ **47. What is Latency in LLM systems?**

Time taken to generate response. Reduced using caching, batching, and smaller models.

☒ **48. What is Quantization?**

Reducing model precision (FP32 → INT8/4) to reduce memory and increase speed.

☒ **49. What is LoRA?**

Low-Rank Adaptation fine-tunes models by training small adapter layers instead of full weights.

☒ 50. Why LoRA is useful?

It drastically reduces GPU cost while maintaining performance.

(🔥 Continuing remaining **51–100** below — equally important.)

★ SECTION 9 — ADVANCED LLM ENGINEERING

☒ 51. What is Token Limit Problem?

Large inputs exceed context window causing truncation. Solutions include chunking and retrieval.

☒ 52. What is Streaming Response?

Tokens are generated progressively instead of waiting full completion, improving UX.

☒ 53. What is Function Calling in LLM?

Model outputs structured JSON triggering backend functions.

☒ 54. What is Structured Output?

Constrained outputs like JSON enforced via schemas.

☒ 55. What is Memory in LLM Apps?

Storage of conversation or facts across interactions.

☒ **56. Types of Memory?**

Short-term, long-term, vector memory.

☒ **57. What is Prompt Template?**

Reusable structured prompt format.

☒ **58. What is LangChain?**

Framework to build LLM pipelines, chains, agents, and integrations.

☒ **59. What is LangGraph?**

Stateful agent workflow orchestration framework.

☒ **60. What is Evaluation Dataset?**

Benchmark dataset to test model performance.

★ **SECTION 10 — SYSTEM DESIGN (VERY HIGH WEIGHTAGE)**

☒ **61. Design ChatGPT-like system?**

Frontend → API → LLM → memory → moderation → logging.

☒ **62. How reduce LLM cost?**

Caching, smaller models, RAG, batching.

☒ **63. What is Semantic Search?**

Search using meaning instead of keywords.

☒ **64. What is Grounding?**

Connecting model output to verified data sources.

☒ **65. What is Knowledge Cutoff?**

Last training data timestamp of model.

☒ **66. Why hallucination happens technically?**

Model predicts probability, not truth verification.

☒ **67. Difference GPT vs BERT?**

GPT is generative autoregressive; BERT is bidirectional encoder.

☒ **68. Encoder vs Decoder?**

Encoder understands input; decoder generates output.

☒ **69. What is Cross Attention?**

Decoder attends encoder outputs in seq2seq models.

☒ **70. What is Positional Encoding?**

Adds order information to tokens.

★ **SECTION 11 — PRACTICAL INTERVIEW QUESTIONS**

☒ **71. How improve chatbot accuracy?**

RAG + prompt engineering + evaluation loop.

☒ **72. How handle long documents?**

Chunking + hierarchical retrieval.

☒ **73. How ensure privacy?**

Local embeddings + secure storage.

☒ **74. What is Token Cost Optimization?**

Reduce prompt size and unnecessary context.

☒ **75. How debug LLM output?**

Log prompts, retrieved docs, and reasoning chain.

☒ **76. What is Evaluation Loop?**

Continuous testing + feedback improvement.

☒ **77. Difference Fine-tuning vs Prompting?**

Weights change vs instructions change.

☒ **78. When to Fine-tune?**

Style learning or domain-specific reasoning.

☒ **79. What is Open-source LLM?**

Models publicly available for modification.

☒ **80. Closed vs Open models?**

API-based vs self-hosted.

 SECTION 12 — REAL INDUSTRY QUESTIONS

☒ **81. Biggest challenge in LLM apps?**

Hallucination, latency, cost, evaluation.

☒ **82. How build production RAG?**

Pipeline + monitoring + feedback loop.

☒ **83. What is Observability?**

Monitoring prompts, latency, accuracy.

☒ **84. What is Prompt Versioning?**

Tracking prompt changes like code versions.

☒ **85. What is AI Safety?**

Prevent harmful or biased outputs.

☒ **86. What is Red Teaming?**

Testing model vulnerabilities intentionally.

☒ **87. Explain LLM inference.**

Forward pass generating tokens sequentially.

☒ **88. GPU vs CPU inference?**

GPU faster due to parallel computation.

☒ **89. What is batching?**

Processing multiple requests together.

☒ **90. What is caching?**

Reuse previous responses or embeddings.

★ **SECTION 13 — FINAL POWER QUESTIONS (INTERVIEW IMPRESS)**

☒ **91. How would you design enterprise chatbot?**

RAG + authentication + audit logs.

☒ **92. How evaluate hallucination?**

Fact-check datasets + human scoring.

☒ **93. Future of LLM?**

Agentic AI and multimodal reasoning.

☒ **94. What is Multimodal LLM?**

Handles text, image, audio together.

☒ **95. What is Toolformer concept?**

Model learns tool usage automatically.

☒ **96. What is Context Engineering?**

Optimizing information inside prompts.

☒ **97. What is Retrieval Quality?**

Accuracy of retrieved context.

☒ **98. Why embeddings dimension matters?**

Higher dimensions capture richer semantics.

☒ **99. What skills required for LLM Engineer?**

ML + NLP + system design + APIs + deployment.

☒ **100. Explain full LLM application pipeline.**

Data ingestion → embeddings → retrieval → prompt construction → generation → evaluation → monitoring → improvement loop.



**ADVANCED LLM ENGINEERING —
NEW INTERVIEW SET (101–150)**

★ SECTION 1 — MODERN LLM ARCHITECTURE (Advanced)

☒ 101. What is Mixture of Experts (MoE) in LLMs?

Mixture of Experts is an architecture where only a subset of model layers (experts) activate for each token instead of the entire network. A routing mechanism selects which experts process the input. This reduces computation cost while allowing massive parameter scaling. Models like Mixtral use MoE to achieve high performance efficiently.

☒ 102. Dense Model vs MoE Model?

Dense models activate all parameters for every token, making them computationally expensive. MoE models activate only selected experts, improving efficiency and scalability. Dense models are simpler and stable, while MoE models require complex routing and load balancing.

☒ 103. What is Speculative Decoding?

Speculative decoding speeds up inference using a smaller draft model that predicts tokens first. A larger model verifies those predictions in batches. If accepted, generation becomes significantly faster. This reduces latency in production LLM systems.

☒ 104. What is KV Cache in LLM inference?

Key-Value cache stores attention computations from previous tokens during generation. Instead of recomputing attention repeatedly, cached values are reused. This drastically speeds up autoregressive inference and reduces compute cost.

☒ 105. Why inference is more expensive than training sometimes?

Inference happens repeatedly for millions of users with long context windows. Each generated token requires forward passes through large models. Real-time latency constraints also require optimized hardware and scaling infrastructure.

★ SECTION 2 — CONTEXT & MEMORY (Very Hot Topic)

☒ 106. What is Long Context Handling in LLMs?

Long-context handling allows models to process large documents or conversations. Techniques include sliding window attention, RAG retrieval, and memory compression. Modern models extend context windows to hundreds of thousands of tokens.

☒ 107. What is Context Compression?

Context compression summarizes or filters input before sending it to the LLM. This reduces token usage while preserving important information. Methods include summarization chains and embedding-based filtering.

☒ 108. What is Retrieval Re-ranking?

After retrieving candidate documents, a reranker model scores them again for relevance. This improves final context quality before generation. Cross-encoder models are commonly used.

☒ 109. What is Lost-in-the-Middle Problem?

LLMs often pay less attention to information located in the middle of long contexts. Important details may be ignored. Solutions include chunk prioritization and structured prompting.

☒ 110. What is Memory-Augmented LLM?

It combines LLM reasoning with external persistent memory systems like vector databases or knowledge graphs. This enables long-term personalization and learning without retraining.

★ SECTION 3 — ADVANCED RAG (VERY HIGH INTERVIEW FREQUENCY)

☒ 111. What is Advanced RAG?

Advanced RAG enhances basic retrieval using reranking, query rewriting, hybrid search, metadata filtering, and multi-step reasoning. It improves accuracy for enterprise applications.

☒ 112. What is Query Rewriting in RAG?

The LLM reformulates user queries into optimized search queries. This improves retrieval relevance, especially for ambiguous questions.

☒ 113. What is Multi-hop Retrieval?

Multi-hop retrieval retrieves information across multiple documents sequentially. Each step uses previous results to refine search. Useful for reasoning-heavy tasks.

☒ 114. What is Contextual Chunking?

Chunks are created based on semantic boundaries rather than fixed token sizes. This preserves meaning and improves retrieval accuracy.

☒ 115. What is Retrieval Evaluation?

Measures retrieval quality using metrics like Recall@K, Precision@K, and MRR. Good retrieval is critical because LLM output depends heavily on retrieved context.

★ SECTION 4 — AGENT ENGINEERING (2025 TREND)

☒ 116. What is Planning in AI Agents?

Planning allows agents to break complex goals into smaller executable steps. The LLM generates a task plan before execution. This improves reliability in multi-step workflows.

☒ 117. What is Tool Selection Strategy?

The agent evaluates available tools and chooses the most relevant based on task description. Proper tool descriptions significantly improve performance.

☒ 118. What is Reflection in Agents?

Reflection enables agents to analyze their own outputs and correct mistakes. The model reviews results and retries tasks with improved reasoning.

☒ 119. What is Autonomous Agent Loop?

Agent repeatedly performs:

Reason → Act → Observe → Update → Repeat

until goal completion. This enables automation workflows.

☒ 120. What is Agent Failure Mode?

Agents may loop infinitely, misuse tools, hallucinate actions, or misinterpret goals. Guardrails and step limits prevent failures.

★ SECTION 5 — LLM OPTIMIZATION & PERFORMANCE

☒ 121. What is Prompt Caching?

Previously processed prompts are cached so repeated queries reuse outputs or embeddings. This reduces latency and API cost.

☒ 122. What is Token Streaming Optimization?

Tokens are streamed immediately as generated instead of waiting full completion. Improves perceived speed and user experience.

☒ 123. What is Batch Inference?

Multiple requests are processed together in a single GPU pass. This increases throughput in production systems.

☒ 124. What is Model Distillation?

A smaller student model learns behavior from a larger teacher model. It maintains performance while reducing size and cost.

☒ **125. What is Inference Quantization Challenge?**

Extreme quantization may degrade reasoning quality. Engineers must balance performance and accuracy carefully.

★ SECTION 6 — LLM SECURITY (Very Important 2026 Interviews)

☒ **126. What is Data Leakage in LLM?**

Sensitive information may appear in outputs due to training data memorization or prompt injection. Proper filtering and isolation are required.

☒ **127. What is Jailbreaking?**

Users manipulate prompts to bypass safety restrictions. Defensive prompting and safety classifiers help mitigate this.

☒ **128. What is Output Moderation?**

Generated responses are checked using safety models before being shown to users.

☒ **129. What is Sandbox Execution?**

External tools run in isolated environments to prevent malicious actions triggered by LLM agents.

☒ 130. Why security is critical in LLM apps?

LLMs interact directly with users and tools, creating risks like data exposure, misinformation, or automated misuse.

★ SECTION 7 — REAL PRODUCTION SYSTEM DESIGN

☒ 131. How design scalable LLM backend?

Load balancer → API layer → inference service → vector DB → cache → monitoring system.

☒ 132. What is Observability in LLM systems?

Monitoring prompts, latency, token usage, failures, and user feedback to improve system performance.

☒ 133. What is Prompt Versioning?

Tracking prompt changes similar to code versioning to measure performance improvements.

☒ 134. What is A/B Testing in LLM apps?

Different prompts or models are tested on users to compare performance metrics.

☒ 135. What is Evaluation Feedback Loop?

User feedback improves prompts, retrieval, and agent behavior iteratively.

★ SECTION 8 — MULTIMODAL LLM (NEW INTERVIEW TREND)

☒ 136. What is Multimodal LLM?

Models capable of understanding text, image, audio, or video together. They align representations across modalities.

☒ 137. How image understanding works in multimodal models?

Images are converted into embeddings via vision encoders and combined with text embeddings inside transformers.

☒ 138. What is Vision Transformer (ViT)?

ViT splits images into patches treated like tokens and processes them using transformer attention.

☒ 139. What is Cross-modal Attention?

Allows interaction between image and text features during reasoning.

☒ 140. Real use cases of multimodal LLM?

Medical analysis, document understanding, visual assistants, robotics perception.

★ SECTION 9 — FUTURE LLM ENGINEERING (HIGH IMPACT QUESTIONS)

☑ 141. What is Small Language Model (SLM)?

Compact models optimized for local deployment and edge devices. They trade scale for efficiency.

☑ 142. Why SLMs are trending?

Lower cost, privacy benefits, and on-device inference requirements.

☑ 143. What is On-device AI?

Running models locally without cloud dependency, improving latency and privacy.

☑ 144. What is Retrieval-Free Knowledge Updating?

Techniques like continual learning update models without full retraining.

☑ 145. What is Synthetic Data Generation?

LLMs generate training data to improve smaller models or specialized tasks.

★ SECTION 10 — SENIOR-LEVEL INTERVIEW QUESTIONS

☒ **146. How evaluate LLM reasoning ability?**

Use benchmark datasets, chain-of-thought evaluation, and human judgment scoring.

☒ **147. How would you debug a failing RAG system?**

Check embedding quality → retrieval relevance → chunk size → prompt structure → generation parameters.

☒ **148. Biggest challenge in production LLM systems?**

Balancing cost, latency, accuracy, and safety simultaneously.

☒ **149. When should you fine-tune instead of RAG?**

When behavior/style learning is required rather than knowledge updating.

☒ **150. What skills define a strong LLM Engineer?**

Understanding transformers, prompt engineering, RAG, agents, system design, optimization, and real deployment experience.

☒ **You Now Have:**

- ✓ 100 Core LLM Interview Q&A
- ✓ 50 Advanced NEW Questions (Latest Industry Level)

👉 **Total = 150 Interview Questions**

SCENARIO-BASED LLM INTERVIEW QUESTIONS (151–200)

SECTION 1 — RAG SYSTEM DESIGN SCENARIOS

☒ **151. Your RAG system gives wrong answers even though documents exist. What will you check?**

First, I would verify embedding quality and ensure correct embedding model selection. Then check chunk size and overlap because poor chunking harms retrieval. Next, analyze retrieval results using similarity scores. I would also test query rewriting and reranking. Finally, inspect prompt structure to ensure retrieved context is actually used by the LLM.

☒ **152. Users complain about hallucinations in your chatbot. How do you fix it?**

I would restrict answers strictly to retrieved context using grounding prompts. Adding citation-based responses reduces hallucination risk. I'd improve retrieval precision and introduce confidence thresholds. If context is insufficient, the system should respond with "information not found."

☒ **153. How would you design a PDF Q&A system?**

Pipeline: document ingestion → chunking → embeddings → vector database storage → retrieval → prompt assembly → LLM generation. I would add metadata filtering and reranking for accuracy. Caching frequent queries improves performance.

☒ **154. RAG answers are slow. How do you optimize latency?**

Use embedding caching, smaller reranker models, and batch retrieval queries. Reduce context tokens through compression. Enable streaming responses and optimize vector DB indexing.

☒ **155. Documents update daily. How will you handle updates?**

Use incremental indexing instead of rebuilding embeddings. Track document versioning and update only changed chunks. Maintain background ingestion pipelines.

SECTION 2 — PROMPT ENGINEERING SCENARIOS

☒ **156. Same prompt gives inconsistent outputs. Why?**

LLMs are probabilistic systems affected by temperature and sampling. I would reduce temperature, add structured instructions, and include examples (few-shot prompting) for stability.

☒ **157. Model ignores instructions sometimes. Fix?**

Move critical instructions near the end or system prompt. Use explicit formatting rules and reinforce constraints multiple times. Structured outputs help enforce compliance.

☒ **158. How ensure structured JSON output?**

Use schema-based prompting with examples. Validate output using parsers and retry generation automatically if format fails.

☒ **159. User prompts break system rules. Solution?**

Use input validation and guardrails before sending prompts to the model. Add moderation layers and instruction hierarchy.

☒ **160. Prompt becomes too long. What will you do?**

Summarize previous conversation, compress context, or retrieve only relevant history dynamically.

SECTION 3 — AGENT SYSTEM SCENARIOS (VERY IMPORTANT)

☒ **161. Agent keeps looping infinitely. How fix?**

Introduce iteration limits and reflection checks. Add success criteria validation and stopping conditions.

☒ **162. Agent selects wrong tools repeatedly.**

Improve tool descriptions and add tool-selection reasoning prompts. Logging decisions helps debugging.

☒ **163. Design an AI research assistant agent.**

Planner module → retrieval tool → summarizer → verifier → final generator. Add memory storage for long tasks.

☒ **164. Agent fails multi-step reasoning tasks.**

Introduce planning phase before execution and allow intermediate reasoning steps.

☒ **165. Agent executes dangerous commands.**

Use sandbox environments and permission-based execution policies.

SECTION 4 — PRODUCTION & SCALING SCENARIOS

☒ **166. API cost suddenly increases. What actions?**

Implement caching, smaller models for simple queries, token limits, and request batching.

☒ **167. Millions of users join your AI app.**

Add load balancing, autoscaling GPUs, queue systems, and asynchronous processing pipelines.

☒ **168. High latency during peak hours.**

Use request prioritization, streaming output, and distributed inference servers.

☒ **169. Model downtime occurs.**

Fallback to secondary models and cached responses.

☒ **170. How monitor LLM production health?**

Track latency, token usage, hallucination rate, user feedback, and failure logs.

★ SECTION 5 — EVALUATION SCENARIOS

☒ **171. How evaluate chatbot quality?**

Use human evaluation, automated benchmarks, factual accuracy checks, and user satisfaction metrics.

☒ **172. Users say answers feel irrelevant.**

Improve retrieval ranking and introduce query expansion.

☒ **173. Compare two prompts objectively.**

Run A/B testing and measure accuracy, latency, and user engagement.

☒ **174. Model answers confidently but wrong.**

Add uncertainty prompts and require citations.

☒ **175. How measure hallucination rate?**

Compare output claims against trusted knowledge sources.

★ **SECTION 6 — MULTIMODAL SCENARIOS**

☒ **176. Build image + text assistant.**

Use vision encoder → shared embedding → multimodal transformer → response generator.

☒ **177. Model misunderstands images.**

Improve preprocessing and provide visual grounding prompts.

☒ **178. Extract information from scanned documents.**

Use OCR pipeline followed by LLM reasoning.

☒ **179. Build resume analyzer using LLM.**

Parse resume → extract entities → evaluate skills → generate feedback.

☒ **180. Voice assistant architecture?**

Speech-to-text → LLM reasoning → text-to-speech output.

★ SECTION 7 — SECURITY SCENARIOS

☒ **181. Prompt injection attack detected.**

Filter malicious instructions and isolate retrieved data from user prompts.

☒ **182. Sensitive data leakage risk.**

Mask confidential information and apply access control layers.

☒ **183. User asks illegal instructions.**

Use moderation APIs and refusal templates.

☒ **184. Prevent jailbreak attempts.**

Use layered prompts, safety classifiers, and output validation.

☒ **185. Secure enterprise chatbot.**

Add authentication, role-based access, and encrypted storage.

★ SECTION 8 — REAL ENGINEERING THINKING

☒ **186. When NOT to use LLM?**

When deterministic logic or simple rules solve the problem cheaper and faster.

☒ **187. LLM gives long unnecessary answers.**

Control verbosity using prompt constraints and token limits.

☒ **188. Improve reasoning accuracy.**

Use chain-of-thought prompting and verification steps.

☒ **189. Users ask follow-up questions frequently.**

Maintain conversation memory and contextual retrieval.

☒ **190. Reduce token cost in chat apps.**

Store summaries instead of full history.

★ SECTION 9 — SENIOR LEVEL SYSTEM DESIGN

☒ 191. Design ChatGPT-like system.

Frontend → API → orchestration layer → retrieval → LLM → memory → monitoring.

☒ 192. Design coding assistant.

Code retrieval + syntax tools + execution validation + LLM reasoning.

☒ 193. Enterprise knowledge assistant design?

Secure RAG + role filtering + audit logging.

☒ 194. AI customer support automation?

Intent detection → retrieval → LLM response → escalation fallback.

☒ 195. Multi-language chatbot?

Detect language → translate → process → respond → translate back.

★ SECTION 10 — TOP INTERVIEW FINISHER QUESTIONS

☒ **196. Biggest limitation of LLM today?**

Lack of true reasoning and dependence on training data patterns.

☒ **197. Future of LLM engineering?**

Agentic systems, multimodal AI, smaller efficient models.

☒ **198. Difference between AI Engineer and LLM Engineer?**

AI Engineer builds ML systems broadly; LLM Engineer specializes in language model applications and orchestration.

☒ **199. One improvement you'd add to LLM systems?**

Reliable factual verification layer integrated with generation.

☒ **200. How do you stay updated in GenAI?**

Research papers, open-source projects, experimentation, and real deployments.

 **PHASE-FINAL PREPARATION — LLM
ENGINEER INTERVIEW**

★ PART 1 — SYSTEM DESIGN

QUESTIONS (MOST IMPORTANT)

☒ Q1. Design a ChatGPT-like System.

☒ English Answer

A ChatGPT-like system consists of multiple layers. First, the user sends a query through a frontend interface. The request goes to an API gateway which handles authentication and routing. Then an orchestration layer prepares prompts, retrieves memory or documents if needed, and sends the request to an LLM. The LLM generates a response which is validated for safety and formatting before being returned to the user. Logging and monitoring systems track performance and usage.

☒ Hinglish Answer

ChatGPT system me user frontend se question bhejta hai. API gateway request ko manage karta hai. Phir orchestration layer prompt banata hai aur memory ya documents retrieve karta hai. LLM answer generate karta hai. Output safety check ke baad user ko send hota hai. Saath me monitoring aur logging system performance track karta hai.

☒ Q2. Design a RAG (Retrieval Augmented Generation) System.

☒ English Answer

A RAG system combines retrieval with generation. Documents are first ingested, chunked, and converted into embeddings stored in a vector database. When a user asks a question, the query is embedded and similar chunks are retrieved. These retrieved documents are added to the prompt and sent to the LLM to generate grounded answers. This reduces hallucination and improves factual accuracy.

☒ Hinglish Answer

RAG system me pehle documents ko chunks me todkar embeddings banaye jate hain aur vector database me store kiya jata hai. User query aane par similar chunks retrieve hote hain. Ye context prompt me add hota hai aur LLM accurate answer deta hai. Isse hallucination kam hota hai.

☒ Q3. How would you scale an LLM application for millions of users?

☒ English Answer

Scaling requires load balancing, distributed inference servers, and asynchronous request processing. Caching frequent responses reduces computation. Autoscaling infrastructure dynamically increases resources during peak traffic. Smaller models can handle simple queries while larger models are reserved for complex tasks.

☒ Hinglish Answer

Scaling ke liye load balancer use karte hain, multiple servers par model run karte hain. Frequent queries cache karte hain. Traffic badhne par auto-scaling servers add ho jate hain. Simple queries small model handle karega aur complex queries large model.

☒ Q4. How do you reduce hallucinations in production LLM systems?

☒ English Answer

Hallucinations can be reduced using RAG grounding, strict prompting, citation-based responses, and confidence thresholds. The system should refuse answers when context is insufficient. Evaluation pipelines continuously monitor factual accuracy.

☒ Hinglish Answer

Hallucination kam karne ke liye RAG use karte hain, strong prompt dete hain, citation-based answers generate karte hain. Agar data nahi mile to model ko “I don’t know” bolna chahiye. Regular evaluation bhi zaroori hai.

★ PART 2 — PROJECT EXPLANATION QUESTIONS (VERY VERY IMPORTANT)

☒ Q5. Explain one LLM project you built.

☒ English Answer

I built an AI assistant using a RAG architecture. The system ingests documents, generates embeddings, and stores them in a vector database. When users ask questions, relevant context is retrieved and passed to the LLM for accurate responses. I implemented prompt engineering, memory handling, and API integration using LangChain and Streamlit.

☒ Hinglish Answer

Maine ek AI assistant banaya jo RAG architecture use karta hai. Documents se embeddings banakar vector database me store kiya. User question par relevant data retrieve karke LLM ko diya jata hai. LangChain aur Streamlit se prompt engineering aur API integration kiya.

☒ Q6. What challenges did you face while building your project?

☒ English Answer

The main challenge was inaccurate retrieval results. I improved chunk size, added overlap, and optimized embedding models. Another issue was latency, which I solved using caching and optimized prompts.

☒ Hinglish Answer

Sabse bada problem retrieval accuracy tha. Chunk size aur overlap improve kiya aur better embedding model use kiya. Latency issue ko caching aur optimized prompt se solve kiya.

☒ Q7. How did you evaluate your LLM application?

☒ English Answer

I evaluated performance using response relevance, factual correctness, latency measurement, and user feedback. I also tested edge cases and compared outputs across multiple prompts.

☒ Hinglish Answer

Evaluation ke liye relevance, accuracy, latency aur user feedback check kiya. Different prompts compare kiye aur edge cases test kiye.

★ PART 3 — DEBUGGING & PRACTICAL THINKING

☒ **Q8. Retrieval is failing even though data exists. What will you do?**

☒ **English Answer**

I would inspect embedding quality, verify chunking strategy, check similarity thresholds, and analyze retrieved documents. Query rewriting and reranking models can improve retrieval accuracy.

☒ **Hinglish Answer**

Embedding model check karunga, chunk size verify karunga, similarity score analyze karunga. Query rewriting aur reranking use karke retrieval improve karunga.

☒ **Q9. LLM responses are too slow. How do you fix latency?**

☒ **English Answer**

Latency can be reduced using response streaming, caching results, reducing token length, and using smaller models when possible. Infrastructure optimization also helps.

☒ **Hinglish Answer**

Streaming enable karenge, cache use karenge, token size kam karenge aur simple queries ke liye small model use karenge. Server optimization bhi help karta hai.

☒ Q10. API cost becomes very high. What steps will you take?

☒ English Answer

I would reduce token usage, summarize conversation history, cache repeated responses, and route simple queries to cheaper models. Monitoring token consumption helps control costs.

☒ Hinglish Answer

Token usage kam karunga, chat history summarize karunga, repeated answers cache karunga aur cheap model simple queries ke liye use karunga.

★ PART 4 — SENIOR THINKING QUESTIONS

☒ Q11. When should you NOT use an LLM?

☒ English Answer

LLMs should not be used when deterministic logic or rule-based systems can solve the problem faster, cheaper, and more reliably. Overusing LLMs increases cost and complexity.

☒ Hinglish Answer

Jab simple rule-based logic kaam kar sakta ho tab LLM use nahi karna chahiye. LLM unnecessary cost aur complexity badha deta hai.

☒ Q12. What makes a good LLM Engineer?

☒ English Answer

A good LLM engineer understands model limitations, prompt design, retrieval systems, evaluation methods, and production deployment. Strong debugging and system design skills are essential.

☒ Hinglish Answer

Achha LLM engineer model limitations samajhta hai, prompt design, RAG systems, evaluation aur deployment janta hai. Debugging aur system design skills bahut important hain.



MOCK INTERVIEW ROUND — LLM ENGINEER (TOP 15)

☒ Q1. Tell me about yourself (AI / LLM Engineer version)

☒ English Answer

I am an AI and Generative AI enthusiast currently focusing on building real-world LLM applications. I have worked on projects involving RAG systems, AI assistants, and Streamlit-based applications using LangChain and modern LLM APIs. My interest lies in solving practical problems using AI rather than only studying theory. I enjoy designing intelligent systems that combine retrieval, reasoning, and automation. Currently, I am strengthening my skills in LLM engineering and scalable AI system design.

☒ Hinglish Answer

Main AI aur Generative AI me focused hoon aur real-world LLM applications bana raha hoon. Maine RAG systems, AI assistants aur Streamlit projects banaye hain LangChain aur APIs ke saath. Mujhe theory se jyada practical problem solving pasand hai.

Intelligent systems design karna mera main interest hai. Abhi main LLM engineering aur scalable AI systems par focus kar raha hoon.

☒ Q2. Explain LLM in simple terms.

☒ English Answer

A Large Language Model is a neural network trained on massive text data to understand and generate human-like language. Instead of memorizing answers, it learns patterns between words and context. This allows it to perform tasks like question answering, summarization, coding assistance, and reasoning through prompts.

☒ Hinglish Answer

LLM ek bada neural network hota hai jo huge text data par train hota hai language samajhne ke liye. Ye answers yaad nahi karta, balki patterns seekhta hai. Isliye ye question answering, summarization aur coding help jaise tasks kar sakta hai.

☒ Q3. Difference between Fine-tuning and RAG?

☒ English Answer

Fine-tuning updates model weights using new training data, making knowledge permanent inside the model. RAG does not change the model; instead, it retrieves external documents during inference and provides them as context. Fine-tuning is expensive but useful for behavior changes, while RAG is flexible and ideal for dynamic knowledge.

☒ Hinglish Answer

Fine-tuning me model ko dubara train karke knowledge permanently add karte hain. RAG me model change nahi hota — external documents runtime par retrieve hote hain. Fine-tuning costly hai, RAG flexible aur practical hai.

☒ Q4. What is hallucination in LLMs?

☒ English Answer

Hallucination occurs when an LLM generates confident but incorrect information. This happens because the model predicts probable text rather than verifying facts. It can be reduced using RAG, grounding prompts, citations, and validation mechanisms.

☒ Hinglish Answer

Hallucination tab hota hai jab model confident but galat answer deta hai. Model prediction karta hai, fact check nahi karta. RAG aur grounding prompts se isko reduce karte hain.

☒ Q5. Explain RAG pipeline step-by-step.

☒ English Answer

First, documents are collected and split into smaller chunks. These chunks are converted into embeddings and stored in a vector database. When a user query arrives, it is embedded and matched against stored vectors. Relevant chunks are retrieved and inserted into the prompt before sending it to the LLM for grounded answer generation.

☒ Hinglish Answer

Pehle documents ko chunks me todte hain. Fir embeddings bana kar vector DB me store karte hain. User query aane par similar chunks retrieve hote hain aur prompt me add hote hain. LLM accurate answer generate karta hai.

☒ Q6. Why do we need embeddings?

☒ English Answer

Embeddings convert text into numerical vectors that capture semantic meaning. This allows machines to compare similarity between texts mathematically. Without embeddings, semantic search and RAG systems would not work effectively.

☒ Hinglish Answer

Embeddings text ko numbers me convert karte hain jisme meaning capture hota hai. Isse semantic similarity calculate hoti hai. RAG aur search systems embeddings ke bina possible nahi.

☒ Q7. How do you improve RAG accuracy?

☒ English Answer

Accuracy can be improved by optimizing chunk size, using better embedding models, adding overlap between chunks, applying reranking models, and improving prompt instructions. Metadata filtering also helps retrieve relevant context.

☒ Hinglish Answer

Chunk size optimize karte hain, better embedding model use karte hain, overlap add karte hain aur reranking apply karte hain. Prompt improvement aur metadata filtering bhi accuracy badhata hai.

☒ Q8. Explain temperature parameter.

☒ English Answer

Temperature controls randomness in model output. A low temperature produces deterministic and focused answers, while a high temperature increases creativity and variability. For factual tasks, lower temperature is preferred.

☒ Hinglish Answer

Temperature randomness control karta hai. Low temperature = stable answers. High temperature = creative answers. Facts wale tasks me low temperature best hota hai.

☒ Q9. How would you reduce LLM cost?

☒ English Answer

I would reduce token usage, summarize conversation history, cache repeated queries, and route simple requests to smaller models. Monitoring token consumption and optimizing prompts also significantly reduces cost.

☒ Hinglish Answer

Token kam use karunga, history summarize karunga, repeated queries cache karunga aur simple tasks small model se karwaunga. Prompt optimization bhi cost kam karta hai.

☒ Q10. What is an AI Agent?

☒ English Answer

An AI agent is a system where an LLM can reason, plan actions, use external tools, and iterate toward completing a task autonomously. It combines decision-making with execution capabilities.

☒ Hinglish Answer

AI agent ek system hai jahan LLM plan karta hai, tools use karta hai aur task complete karne ke liye steps leta hai automatically.

☒ Q11. Difference between AI Agent and Chatbot?

☒ English Answer

A chatbot mainly responds to user queries, while an AI agent can take actions, call tools, and perform multi-step reasoning. Agents are goal-oriented systems.

☒ Hinglish Answer

Chatbot sirf reply deta hai. AI agent actions leta hai, tools use karta hai aur multi-step tasks complete karta hai.

☒ Q12. Biggest limitation of LLMs?

☒ English Answer

LLMs lack true reasoning and real-world understanding. They rely on statistical patterns rather than verified knowledge, which leads to hallucinations and inconsistencies.

☒ Hinglish Answer

LLM real understanding nahi rakhta. Ye patterns par based hota hai, isliye kabhi galat confident answers deta hai.

☒ Q13. How do you evaluate LLM output?

☒ English Answer

Evaluation includes factual accuracy, relevance, coherence, latency, and user satisfaction. Both automated metrics and human evaluation are important.

☒ Hinglish Answer

Evaluation me accuracy, relevance, response quality aur latency check karte hain. Human feedback bhi important hota hai.

☒ Q14. Explain prompt engineering.

☒ English Answer

Prompt engineering is the process of designing instructions that guide the LLM toward desired outputs. It includes role definition, examples, constraints, and output formatting.

☒ Hinglish Answer

Prompt engineering matlab model ko sahi instruction dena taki desired output mile. Isme role, examples aur formatting define karte hain.

☒ Q15. Why should we hire you for an LLM Engineer role?

☒ English Answer

I combine strong conceptual understanding with practical project experience in LLM applications. I focus on solving real problems using RAG systems, prompt engineering, and scalable AI solutions. I continuously learn and experiment with new GenAI technologies, which helps me adapt quickly to evolving AI systems.

☒ Hinglish Answer

Mere paas concepts + practical project experience dono hai. Maine RAG aur LLM applications banaye hain aur real problems solve kiye hain. Main continuously new GenAI tech learn karta hoon, isliye fast adapt kar sakta hoon.

FINAL STATUS

You now have:

☒ 200 Core + Advanced Questions

✓ Phase-Final Preparation

✓ System Design Answers

✓ Mock Interview Round

✓ Speaking-ready answers (English + Hinglish)

👉 This is **SERIOUS LLM ENGINEER INTERVIEW LEVEL**.